APS360 Progress Report: AI-Generated Music Detection

Lily Chen
1010051694, lilly.chen@mail.utoronto.ca
https://github.com/lil-yc/aps360-project

Brief Project Description

With the rise of generative music models (e.g. Suno, Udio, Riffusion), AI-generated music is becoming indistinguishable from human-composed music. This raises concerns around copyright, misinformation, and artist attribution. The goal of this project is to develop a deep learning model that can classify short music clips as either human-composed or AI-generated. Deep learning is suitable for this problem because it can learn to detect spectral artifacts or lack of long-term structural coherence that traditional heuristics might miss. CNNs and RNNs have demonstrated strong performance on audio classification tasks by learning hierarchical representations directly from time-frequency features. The primary model, a CNN, takes in Mel spectrograms (Figure 1) of AI or human music clips as an input, and outputs a 1 for AI-generated and 0 for human-composed. This report will discuss data processing, a baseline model, and progress made on the primary CNN.

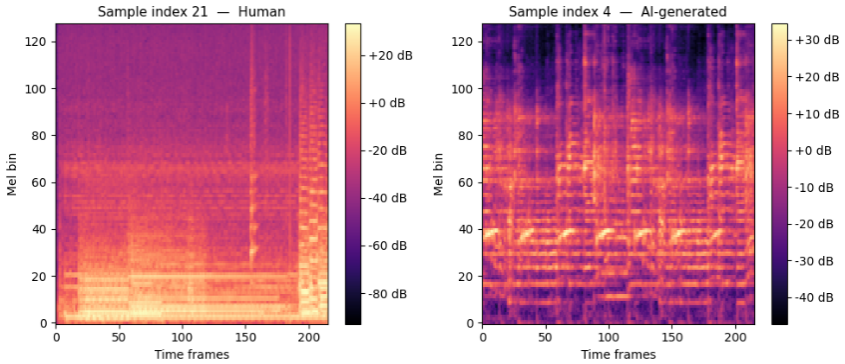

Figure 1: Mel spectrograms of a human vs. AI music clip.

Data Processing

So far, 1000 Human-composed and AI-generated music tracks were collected from this dataset, with an even split of 500 tracks per class. This smaller set was used as a starting point; for future training, there are a total of 13,554 tracks available. After loading the tracks, several forms of audio processing were applied: The tracks were resampled at 22,050 Hz and converted to mono to ensure consistency. The tracks were cut down into 5 second clips so the resulting Mel spectrograms would be a consistent shape.

For the baseline model, Mel frequency cepstral coefficients were computed from each clip, and for the CNN, the clips were converted to Mel spectrograms (see Figure 1 for visualization).

For the baseline model, the data was split 80/20 for training and testing. There wasn't a need for a validation set since the baseline didn't need to be tuned. For the primary model, the data was split 70/15/15 for training, validation, and testing.

Baseline Model

A baseline model was implemented by using the Mel frequency cepstral coefficients (MFCCs) of the audio clips to train a logistic regression classifier. After audio processing, 26 MFCCs were computed from each clip, so the logistic regression model takes only 26 features to fit to a curve. Since the model is a simple baseline, it did not require any tuning, and it was trained for 30 epochs.

As shown by the training loss curve for the baseline, the logistic regression model was able to converge and reach a training loss of 0.57 after 30 epochs, not lowering any further. The model was only able to achieve a test accuracy of 0.67, and average precision, recall, and f1-score of 0.67 as well. The model performs better than random guessing (50%) showing that there are detectable patterns, but the accuracy isn't very high, which shows the limitations of logistic regression and MFCCs for this task.

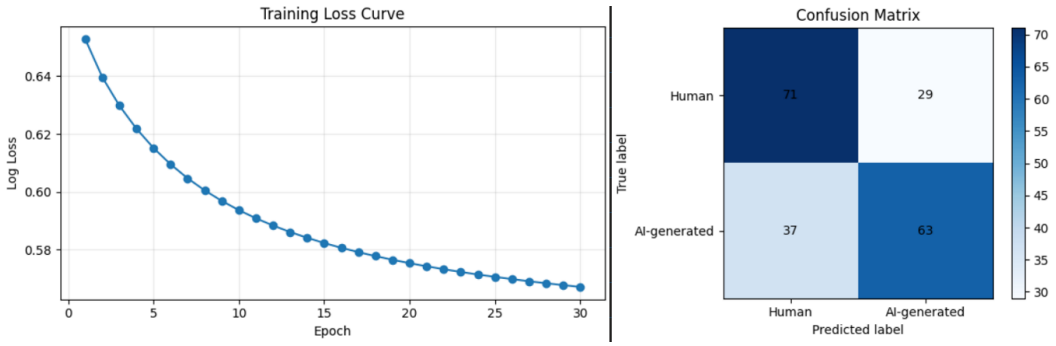

Figure 2: Training loss curve and confusion matrix of baseline model.

From the confusion matrix in Figure 2, there were 63 true positives, 71 true negatives, 29 false positives, and 37 false negatives. There seems to be a bias toward false negatives. A qualitative observation is that the false positives tend to have more processed and electronic sounds, and the false negatives tend to have more acoustic sounds, showing a potential bias with this model.

Primary Model

The primary model (Figure 1) for this project is a CNN taking Mel spectrograms as inputs. The network consists of three convolutional blocks, with batch normalization and ReLU activations, followed by max pooling layers to progressively reduce dimensionality. The extracted features are passed through fully connected layers and a final sigmoid output unit for binary classification. Additionally, dropout regularization is applied, with dropout increasing after every layer to improve generalization. The loss function was binary cross entropy, and the Adam optimizer was used. The learning rate was 0.001, batch size was 16, and the model was trained for 30 epochs.

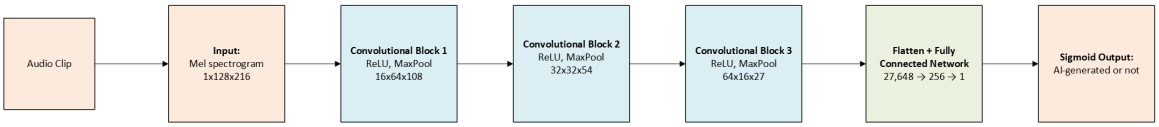

Figure 3: Diagram outlining CNN architecture.

The model was able to converge, as shown by the loss and error curves in Figure 4. It achieved a test accuracy of 0.966, precision 0.974, recall 0.961, and f1-score 0.967. Already, the CNN has a significantly better performance compared to the logistic regression baseline, demonstrating that this architecture is suitable for the task.

As shown by the confusion matrix (Figure 5) the false positives and false negatives are close together. A qualitative observation is that the 2 false positives were both solo drum tracks, and the 3 false negatives were all non-musical sound effects or ambience. This shows a potential difficulty in classifying non-melodic audio.

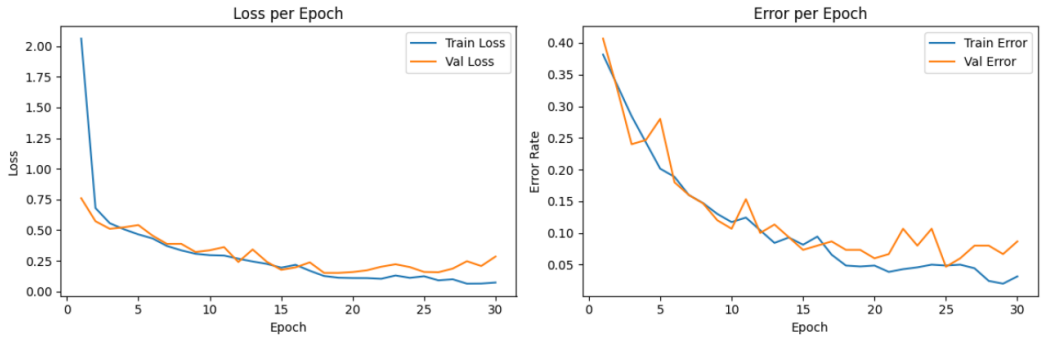

Figure 4: Test and validation loss and error curves of the primary model.

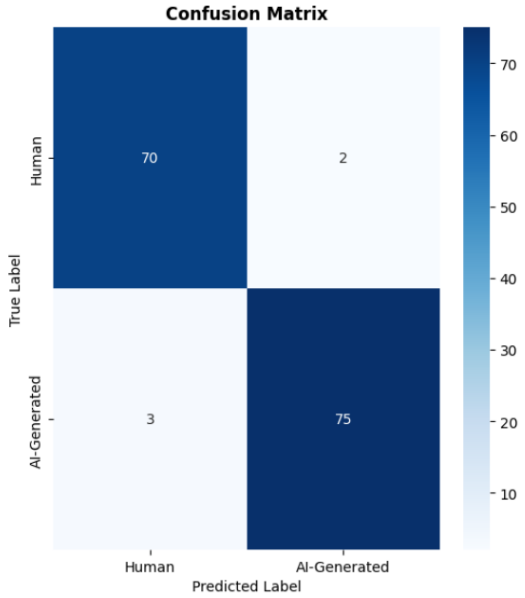

Figure 5: Confusion matrix of the primary model.

A challenge encountered during developing this model was that the training time was very long (hours for 30 epochs), and this issue was addressed by caching the Mel spectrograms that were created.

Next Steps

As observed from the high accuracy, the architecture of the current CNN is likely suitable, but there are some potential data processing oversights in the current pipeline that may lead this model to perform well in testing, but perform poorly in real-world conditions. Currently, only the first 5 seconds of tracks are taken. This may be the cause of the primary model's false negatives, which included only sound effects in the beginning of the track. To mitigate this, in the future, the segment window will be randomly selected, or chosen to keep the most characteristic section of the track.

The size of the test set will also be increased, since the test set is currently only 150 samples, which may lead to an unbalanced representation. A window of only 5 seconds might limit the model's ability to detect patterns so if time permits, the duration window may be increased.

Additionally, the model will be trained and tested on a larger dataset, and hyperparameters such as batch size will be further tuned to attempt to improve the test metrics of the model further, and see if it's possible to reach an accuracy of 0.99, as other similar CNNs were able to achieve.